Simon's Algorithm

An Exponential Quantum Speedup

Rushank Naik

Quantum Algorithms

April 22, 2026

Outline

1. Motivation & Problem Statement
2. Quantum Circuit & Algorithm
3. Worked Example: Reconstructing s
4. Complexity & Classical Comparison
5. Key Takeaways
6. Practice Problem

Why Simon's Algorithm?

Historical Significance

- Proposed by **Daniel Simon (1994)**
- First algorithm to show an **exponential** quantum speedup over classical
- Inspired **Shor's factoring algorithm**
- Belongs to the *hidden subgroup problem* family

Complexity Gap

Model	**Queries**
Classical	$\Omega(2^{n/2})$
Quantum	$O(n)$

Exponential speedup!

Problem Statement

Given

A black-box function $f : \{0,1\}^n \to \{0,1\}^n$

Promise (2-to-1 property)

There exists a **secret string** $s \in \{0,1\}^n$ such that

$$f(x) = f(y) \iff y = x \oplus s$$

Every output value is hit by **exactly two** inputs (x and $x \oplus s$).

- If $s = 0^n$: f is *one-to-one*
- If $s \neq 0^n$: f is *two-to-one*

Goal

Find s using as few queries to f as possible.

Concrete Example: $n = 3$, $s = 101$

Function table

x	$f(x)$
000	000
001	010
010	001
011	100
100	010
101	000
110	100
111	001

Observe the 2-to-1 pairing

Each output appears **exactly twice**:

$$\underbrace{000 \leftrightarrow 101}_{000 \oplus 101 = 101} \qquad \underbrace{001 \leftrightarrow 110}_{001 \oplus 110 = 111\,?}$$

More precisely $f(x) = f(x \oplus s)$, e.g.

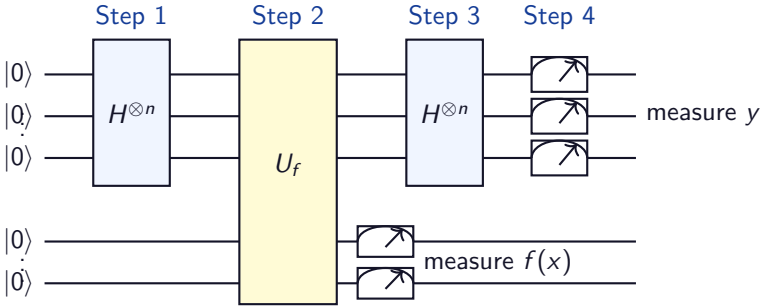
$$f(000) = f(101) = 000$$

$$f(011) = f(110) = 100$$

Classical cost

Birthday paradox: need $\approx \sqrt{2^n} = 2^{n/2}$ queries — *exponential* in n.

Quantum Circuit Overview



Step 1: $H^{\otimes n}$ on $|0\rangle^n$ **Step 2:** Apply U_f **Step 3:** $H^{\otimes n}$ on 1st reg **Step 4:** Measure 1st reg

Step-by-Step State Analysis

Step 1 — Hadamard on first register:

$$|0\rangle^{\otimes n} \xrightarrow{H^{\otimes n}} \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle\, |0\rangle^{\otimes n}$$

Step 2 — Apply oracle U_f:

$$\frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle\, |f(x)\rangle$$

Step 3 — Measure second register (see $f(r)$):

The first register collapses to a **uniform superposition** over the two pre-images:

$$\frac{1}{\sqrt{2}}\big(|r\rangle + |r \oplus s\rangle\big) \quad \text{where } r \text{ is a random } n\text{-bit string}$$

Step 4 — Second Hadamard on first register:

$$H^{\otimes n} \frac{1}{\sqrt{2}}\big(|r\rangle + |r \oplus s\rangle\big) = \frac{1}{2^{(n-1)/2}} \sum_{\substack{y \in \{0,1\}^n \\ y \cdot s \equiv 0 \pmod 2}} (-1)^{y \cdot r}\, |y\rangle$$

Fourier Sampling — Getting a Random y

What we measure

After the second Hadamard + measurement we obtain a uniformly random y satisfying:

$$\boxed{y \cdot s \equiv 0 \pmod 2} \iff y_1 s_1 + y_2 s_2 + \cdots + y_n s_n \equiv 0 \pmod 2$$

The Algorithm

1. **Set up superposition:** $\frac{1}{\sqrt{2}}|r\rangle + \frac{1}{\sqrt{2}}|r \oplus s\rangle$
2. **Fourier sample** to get a random y with $y \cdot s = 0 \pmod 2$
3. **Repeat $n-1$ times** to collect $n-1$ linearly independent equations in s
4. **Solve** the linear system over $\mathbb{F}_2$ (Gaussian elimination)

Key Point

Each quantum run gives ONE linear equation. We need $n-1$ independent ones to uniquely determine s.

Worked Example — Setup ($n = 3$, $s = 101$)

Setting

$n = 3$, secret string $s = 101$

We need $n - 1 = 2$ linearly independent measurements $y^{(1)}, y^{(2)}$ satisfying

$$y \cdot s = 0 \pmod 2$$

i.e. $y_1 s_1 + y_2 s_2 + y_3 s_3 = 0 \pmod 2$

All valid y values for $s = 101$

$$y \cdot 101 = y_1 \cdot 1 + y_2 \cdot 0 + y_3 \cdot 1 = 0$$

Solutions: $y_1 + y_3 \equiv 0 \pmod 2$, so

$$y \in \{000,\ 010,\ 101,\ 111\}$$

Quantum circuit samples *uniformly* from this set.

Worked Example — Collecting Equations

Suppose the circuit returns:

$$y^{(1)} = 101, \qquad y^{(2)} = 111$$

Write the linear system over $\mathbb{F}_2$:

$$\begin{cases} 1 \cdot s_1 + 0 \cdot s_2 + 1 \cdot s_3 = 0 \pmod 2 & \leftarrow y^{(1)} = 101 \\ 1 \cdot s_1 + 1 \cdot s_2 + 1 \cdot s_3 = 0 \pmod 2 & \leftarrow y^{(2)} = 111 \end{cases}$$

Subtract (XOR) eq. 1 from eq. 2:

$$(1 \oplus 1)\, s_1 + (0 \oplus 1)\, s_2 + (1 \oplus 1)\, s_3 = 0 \;\Rightarrow\; s_2 = 0$$

Substitute $s_2 = 0$ into eq. 1:

$$s_1 + s_3 = 0 \;\Rightarrow\; s_1 = s_3$$

Worked Example — Solving for s

Two solutions over $\mathbb{F}_2$

From $s_2 = 0$ and $s_1 = s_3$:

$$s \in \{000,\ 101\}$$

- $s = 000$ is the *trivial* (identity) solution — means f is 1-to-1.
- $s = 101$ is the **non-trivial secret string**!

Verification

Check: $f(000) = f(000 \oplus 101) = f(101) = 000$ ✓
Check: $f(011) = f(011 \oplus 101) = f(110) = 100$ ✓

Conclusion

With only **2 quantum measurements** (vs. $\Omega(2^{n/2})$ classically), we recovered $s = \mathbf{101}$.

Classical vs. Quantum Complexity

Classical Algorithm

- Must query f and look for a *collision*: $f(x) = f(y)$
- Birthday paradox: need $\approx \sqrt{2^n}$ queries

$$\sqrt{N} = 2^{n/2} \quad \text{(exponential in } n\text{)}$$

- No clever structure can beat this lower bound

Simon's Quantum Algorithm

- Repeat circuit $O(n)$ times to get $n-1$ equations
- Each run: 1 quantum query + 1 measurement
- Gaussian elimination: $O(n^3)$ classical post-processing
- **Total: $O(n)$ quantum queries**

Why Does This Work? — Intuition

Quantum Parallelism

The Hadamard gate creates a **uniform superposition** over all 2^n inputs simultaneously. One oracle call processes all of them at once.

Interference

The second $H^{\otimes n}$ causes **constructive interference** only for y with $y \cdot s = 0$ and **destructive interference** for all others. Measurement *collapses* to one such y.

Linear Algebra over $\mathbb{F}_2$

Each run contributes one constraint. After $n-1$ independent constraints, **Gaussian elimination** over $\mathbb{F}_2$ uniquely recovers s (up to the trivial $s = 0^n$).

Connection to Shor's Algorithm

Simon's Algorithm is a **direct precursor** to Shor's (1994 $\rightarrow$ 1995):

- Both solve a **hidden subgroup problem (HSP)**
- Simon: HSP over $(\mathbb{Z}_2)^n$
- Shor: HSP over $\mathbb{Z}_N$
- Both use **Fourier sampling** + classical post-processing
- Shor adds the Quantum Fourier Transform (QFT)

Hidden Subgroup Problem

Find the hidden subgroup H of a group G given a function f that is *constant and distinct* on cosets of H.

Simon: $G = (\mathbb{Z}_2)^n$, $H = \{0^n, s\}$

Summary

Step	Description
1	Apply $H^{\otimes n}$ to $\|0\rangle^n \Rightarrow$ uniform superposition
2	Query oracle U_f
3	Measure 2nd register (collapses to superposition $\|r\rangle + \|r \oplus s\rangle$)
4	Apply $H^{\otimes n}$ to 1st register
5	Measure 1st register $\Rightarrow$ get y with $y \cdot s = 0 \pmod 2$
6	Repeat $n-1$ times to collect independent equations
7	Gaussian elimination over $\mathbb{F}_2$ to find s

Key Result

Simon's algorithm uses $O(n)$ quantum queries. Any classical algorithm requires $\Omega(2^{n/2})$ queries.

Question b — Applying Simon's Algorithm ($n = 3$, $s = 111$)

Given function f

x	$f(x)$
000	000
001	010
010	001
011	100
100	100
101	001
110	010
111	000

Period: $a = 111$, so $f(x_i) = f(x_i \oplus 111)$

Question

$f(x)$ is 2-to-1 and the period is $a = 111$.

What is the initial input of Simon's algorithm?

Solution

The secret string is $s = a = \mathbf{111}$.

Simon's algorithm initialises two quantum registers of n qubits, both set to $|0\rangle$:

$$|\psi_0\rangle = |0\rangle^{\otimes 3} \otimes |0\rangle^{\otimes 3} = |000\rangle\,|000\rangle$$

- **1st register:** $|000\rangle$ — input to $f(x)$
- **2nd register:** $|000\rangle$ — stores $f(x)$

Thank You!

Questions?

References

- D. Simon, *On the Power of Quantum Computation*, SIAM J. Comput., 1997.
- Nielsen & Chuang, *Quantum Computation and Quantum Information*, Cambridge.
- Lecture notes: *Simon's Algorithm*, Lecture 8.